RAG: medir

M4: Medir, v0.2



Hoy: el recorrido

De «parece que anda» a «lo medí»



Venís de armar el buscador. Ahora medís si mejora las respuestas.

Paso 1 de 4: Dónde quedamos

Sin mirar: ¿dónde falló tu sistema?



?

¿Recuperación?

no encontró el fragmento



?

¿Generación?

lo encontró y respondió mal

Elegí una pregunta de tu suite que no se resolvió. ¿Cómo lo sabés?

¿Dónde falló: al buscar o al redactar?

Posibles respuestas: buscar o redactar



Recuperación

no llegó el fragmento con la respuesta



Generación

llegó y respondió mal o inventó



Cómo saberlo

rag.py con la misma pregunta muestra qué llegó, sin gastar

Una suite es tu conjunto de prueba

Está en los documentos

prueba que lo encuentre y lo use

No está en ninguno

prueba que diga NO ENCONTRADO

Intenta engañarlo

prueba que no se deje llevar

Siempre las mismas preguntas, como un test set.

Cada número mide una parte distinta

Métrica	Pregunta	Quién la calcula
hit	¿Llegó el documento correcto?	`suite.py`
Abstención correcta	¿Dijo NO ENCONTRADO solo cuando no estaba?	`suite.py`
Fidelidad (faithfulness)	¿La respuesta sale de los fragmentos?	Ragas (opcional)
Correcta	¿Es verdad?	Vos

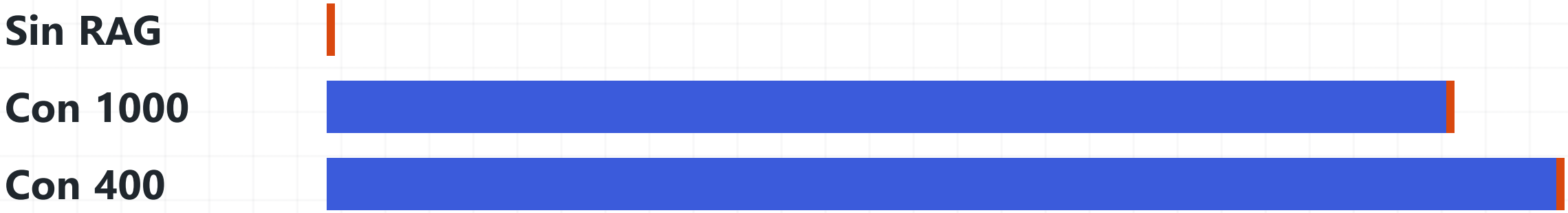
Con RAG subió el hit, pero se abstuvo de más

```
sin RAG: hit  0/11 · abstención correcta 11/13 · latencia mediana 0.8s
con RAG: hit 10/11 · abstención correcta  8/13 · latencia mediana 1.5s

las 5 abstenciones de más: 1 sin el documento              → recuperación
                           3 con el documento, sin fragmento  → recuperación
                           1 con el fragmento (la 13)         → generación
```

- Medido con Gemini 3.5 Flash-Lite, 17/10.
- **hit**: ¿el documento correcto está entre los 4 que trajo? Un *recall@4*, como en ML.
- ¿Qué perilla probarías primero?

Buscar en los documentos cambió todo



- Llegó el documento correcto, de 11: sin RAG 0, con 1000/150 10, con 400/50 11 (fragmento/overlap).
- Sin RAG no busca: su 0 es por definición. Compará sus respuestas.

El hit dio bien, pero el fragmento no servía

Pregunta 7: llega el documento
(1 de los 4 fragmentos),
ninguno dice RemoveMessage

→

Respuestas
Modelo chico: «usá Command»
Gemini: NO ENCONTRADO

- El hit mira el documento, no el fragmento.
- ¿Qué mirarías antes de culpar al modelo?

Mové el tamaño y mirá qué se recupera

M4 · ¿Qué recupera tu índice?

Solo recuperación: embeddings locales, **sin llamadas pagas**. Antes de mover un control, predecí: ¿aparece el documento esperado? En la tabla, **distancia más chica = fragmento más parecido** a la pregunta.

Pregunta de la suite

¿Qué pasa con los checkpoints de MemorySaver cuando el proceso se reinicia?

Tamaño de fragmento (caracteres) 1,000 Overlap 150 k 4

fragmentos

189

esperado: langgraph_persistence

✅ **encontrado**

Search... Columns Explore Export

#	documento	distancia	fragmento
1	langgraph_persistence	0.984	**Fix:** Keep `thread_id` values under 255 character...
2	langgraph_use-subgraphs	1.185	The `checkpointer` parameter on `.compile()` contro...
3	langgraph_persistence	1.225	## Checkpointer vs. store | | Checkpointer | Store | |-...
4	langgraph_persistence	1.232	LangGraph provides two complementary persistenc...

4 rows, 4 columns *No selection*

Con fragmentos más chicos apareció el correcto

Fragmentos de 1000
pregunta #8
❌ **fuera de los 4**

→

Fragmentos de 400
pregunta #8
✅ **4.º de 4: por poco**

#8: ¿qué checkpointer se recomienda en producción? Suite de referencia, 15/10. Con tus documentos, puede ser al revés.

Para otra pregunta ayudó subir K, no achicar

```
¿Qué clase de LangChain borra mensajes de la memoria?

K = 4, CHUNK = 1000: RemoveMessage: no llegó
K = 8, CHUNK = 1000: RemoveMessage: puestos [5, 8]
K = 4, CHUNK =  400: RemoveMessage: no llegó
en inglés, K = 4   : RemoveMessage: puestos [1, 2, 4]
```

Cada perilla ayuda a unas preguntas y no a otras: por eso se mide con toda la suite.

Dos búsquedas y un reordenador mejoran el orden



Híbrida: palabras exactas y significado, juntos.

Es opcional: si lo probás, medilo como una variación más.

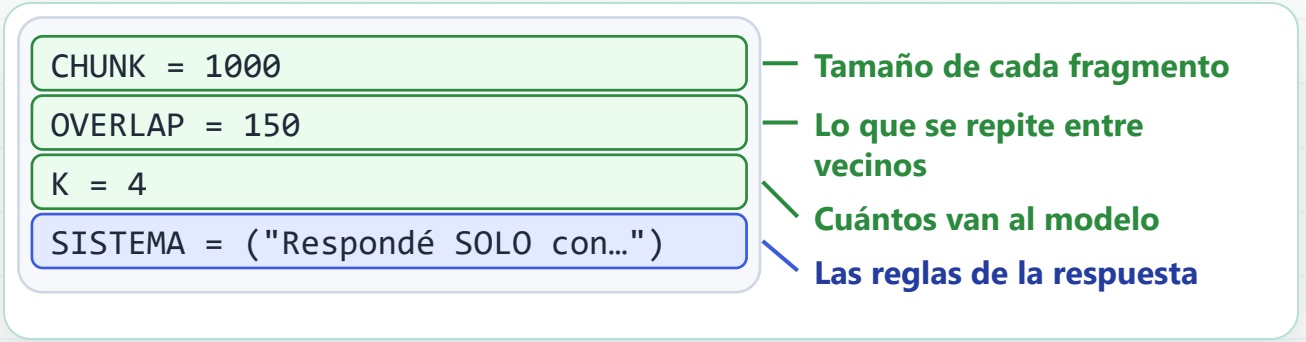
Una corrida por decisión

```
uv run python suite.py v02_rag:sin_rag datos/mi_suite.jsonl --tag sin-rag
uv run python suite.py v02_rag:con_rag datos/mi_suite.jsonl --tag con-rag
```

Mismo modelo y mismas preguntas: cambia una sola cosa por corrida.

Taller B: sin documentos, con documentos (45 min)

1. **Suite** sin y con RAG.
2. **Cada fallo:** ¿recuperación o generación?
3. **Una perilla:** `CHUNK`, `OVERLAP` o `K`



Práctica M4, pasos 5 a 8.

Primer control · 10–15 min: existe `logs/suite-sin-rag.csv`

Antes de irte, anotá dónde quedaste

- **Estoy en:** qué paso del taller B.
- **Ya tengo:** el CSV sin RAG, el CSV con RAG o los dos.
- **El lunes sigo con:** la perilla que voy a cambiar.

Una línea en el chat, para retomar el lunes.

Clínica: una que mejora, una que no

- Una pregunta que mejoró con tus documentos
- Una que no mejoró
- Un límite que no resolviste

Devolución: una fortaleza, un problema y una próxima prueba.

Repaso: de estas dos clases te llevás tres ideas



RAG

buscar antes de responder



Vectores

traer los K más cercanos



Medir

el hit no dice si es verdad

Llegar al documento correcto no alcanza: mirá también la respuesta.

Cierre de v0.2: ¿dónde no ayudó?

¿En qué pregunta buscar en los documentos no ayudó?

¿Qué cambiarías y qué medirías?

Microdefensa: ¿por qué una respuesta que cita un documento puede estar equivocada?

Anotá tu **confianza (1 a 4)**. Entrega de avance 2: `evidencia/v0.2.md` , *push* hasta el mar 10/11, 23:59.